

Task 1: Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory

```

shreyas@Grigori:~$ mkdir -p ~/memory_layout
shreyas@Grigori:~$ cd ~/memory_layout
shreyas@Grigori:~/memory_layout$ nano memory.c
shreyas@Grigori:~/memory_layout$ gcc memory.c -o memory
shreyas@Grigori:~/memory_layout$ ./memory
===== Linux Process Memory Layout =====

```

```

PID                : 647
Code/Text address  : 0x64110ca31209
Global variable    : 0x64110ca34010
Static variable    : 0x64110ca34014
Heap variable      : 0x64113011f2a0
Stack variable     : 0x7fff806e571c

```

```

=====

```

```

Address interpretation:

```

```

Code/Text  -> my_function()

```

```

Global      -> global_var

```

```

Static      -> static_var

```

```

Heap        -> memory allocated by malloc()

```

```

Stack       -> local variable in main()

```

```

shreyas@Grigori:~/memory_layout$ cat memory.c

```

```

#include <stdio.h>

```

```

#include <stdlib.h>

```

```

#include <unistd.h>

```

```

int global_var = 10;

```

```

static int static_var = 20;

```

```

void my_function()

```

```

{

```

```

}

```

```

int main()

```

```

{

```

```

    int stack_var = 30;

```

```

    int *heap_var = (int *)malloc(sizeof(int));

```

```

    if (heap_var == NULL)

```

```

    {

```

```

        *heap_var = 40;

```

```

        printf("===== Linux Process Memory Layout =====\n\n");

```

```

        printf("PID                : %d\n", getpid());

```

```

        printf("Code/Text address  : %p\n", (void *)my_function);

```

```

        printf("Global variable    : %p\n", (void *)&global_var);

```

```

        printf("Static variable    : %p\n", (void *)&static_var);

```

```

        printf("Heap variable      : %p\n", (void *)heap_var);

```

```

        printf("Stack variable     : %p\n", (void *)&stack_var);

```

```

        printf("\n===== \n");

```

```

        printf("\nAddress interpretation:\n");

```

```

        printf("Code/Text  -> my_function()\n");

```

```

        printf("Global      -> global_var\n");

```

```

        printf("Static      -> static_var\n");

```

```

        printf("Heap        -> memory allocated by malloc()\n");

```

```

        printf("Stack       -> local variable in main()\n");

```

```

        free(heap_var);

```

```

        return 0;

```

```

}

```

```

shreyas@Grigori:~/memory_layout$ |

```

Task 2: Make use of the /proc//maps file and memory analysis tools to study the memory organization of a running process. Prepare a report explaining virtual memory mappings.

```
shreyas@Grigori:~/memory_layout$ mkdir -p ~/memory_analysis
shreyas@Grigori:~/memory_layout$ cd ~/memory_analysis
shreyas@Grigori:~/memory_analysis$ nano memory_analysis.c
shreyas@Grigori:~/memory_analysis$ gcc memory_analysis.c -o memory_analysis
shreyas@Grigori:~/memory_analysis$ ./memory_analysis
```

```
PID: 674
Heap address: 0x7bfcd44ff010
Press Enter to exit...
|
```

```
shreyas@Grigori:~$ cat /proc/674/maps
576e6495a000-576e6495b000 r--p 00000000 00:30 12150 /home/shreyas/memory_analysis/memory_analysis
576e6495b000-576e6495c000 r-xp 00001000 00:30 12150 /home/shreyas/memory_analysis/memory_analysis
576e6495c000-576e6495d000 r--p 00002000 00:30 12150 /home/shreyas/memory_analysis/memory_analysis
576e6495d000-576e6495e000 r--p 00003000 00:30 12150 /home/shreyas/memory_analysis/memory_analysis
576e6495e000-576e6495f000 rw-p 00004000 00:30 12150 /home/shreyas/memory_analysis/memory_analysis
576e90e2e000-576e90e4f000 rw-p 00000000 00:00 0 [heap]
7bfcd44ff000-7bfcd4600000 rw-p 00000000 00:00 0
7bfcd4600000-7bfcd4628000 r--p 00000000 00:30 49240 /usr/lib/x86_64-linux-gnu/libc.so.6
7bfcd4628000-7bfcd47b1000 r-xp 00028000 00:30 49240 /usr/lib/x86_64-linux-gnu/libc.so.6
7bfcd47b1000-7bfcd4800000 r--p 001b1000 00:30 49240 /usr/lib/x86_64-linux-gnu/libc.so.6
7bfcd4800000-7bfcd4804000 r--p 001ff000 00:30 49240 /usr/lib/x86_64-linux-gnu/libc.so.6
7bfcd4804000-7bfcd4806000 rw-p 00203000 00:30 49240 /usr/lib/x86_64-linux-gnu/libc.so.6
7bfcd4806000-7bfcd4813000 rw-p 00000000 00:00 0
7bfcd4813000-7bfcd481e000 rw-p 00000000 00:00 0
7bfcd481e000-7bfcd482e000 rw-p 00000000 00:00 0
7bfcd482e000-7bfcd4830000 rw-p 00000000 00:00 0
7bfcd4830000-7bfcd48e5000 rw-p 00000000 00:00 0
7bfcd48e5000-7bfcd48e9000 r--p 00000000 00:00 0 [vvar]
7bfcd48e9000-7bfcd48eb000 r--p 00000000 00:00 0 [vvar_vclock]
7bfcd48eb000-7bfcd48ed000 r-xp 00000000 00:00 0 [vdso]
7bfcd48ed000-7bfcd48ee000 r--p 00000000 00:30 49053 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7bfcd48ee000-7bfcd4919000 r-xp 00001000 00:30 49053 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7bfcd4919000-7bfcd4923000 r--p 0002c000 00:30 49053 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7bfcd4923000-7bfcd4925000 r--p 00036000 00:30 49053 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7bfcd4925000-7bfcd4927000 r--p 00038000 00:30 49053 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7ffe00000000-7ffe0000b1000 rw-p 00000000 00:00 0 [stack]
shreyas@Grigori:~$ |
```

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    int *heap = malloc(1024 * 1024);

    if (heap == NULL)
    {
        return 1;
    }

    printf("PID: %d\n", getpid());
    printf("Heap address: %p\n", (void *)heap);
    printf("Press Enter to exit...\n");

    getchar();

    free(heap);

    return 0;
}
```


